

Rapport de Soutenance Finale : HarmonyStudio

Projet S4 – EPITA

Alexandre d'AVEZAC de CASTERA
Léa-Angéline KOLMERSCHLAG
Hugo GUYENNET
Quentin JOLY

Table des matières

1	Rôles et missions des membres	2
1.1	Alexandre – Moteur DSP & synthèse sonore	2
1.2	Quentin – Séquençage & structures de données temporelles	2
1.3	Léa – Interface graphique	2
1.4	Hugo – Système de fichiers & export audio	2
2	Objectifs, planification et état d’avancement	3
2.1	Objectifs	3
2.2	Planning	3
2.3	État d’avancement réel	4
3	Difficultés, choix technologiques et algorithmiques	5
3.1	Moteur audio temps réel	5
3.1.1	Choix technologiques	5
3.1.2	Choix algorithmiques	5
3.1.3	Évolution : passage en stéréo	5
3.1.4	Difficultés rencontrées	5
3.2	Synthèse sonore	6
3.2.1	Chaîne de signal	6
3.2.2	Génération de formes d’ondes : PolyBLEP	6
3.2.3	Karplus-Strong – modèle physique de corde	6
3.2.4	Enveloppes ADSR	6
3.2.5	Filtrage numérique : filtre biquad	7
3.2.6	Polyphonie et voice stealing	7
3.2.7	Difficultés rencontrées	7
3.3	Effets audio	8
3.3.1	Architecture	8
3.3.2	Delay	8
3.3.3	Réverbération de Schroeder	8
3.3.4	Chorus	8
3.3.5	Difficultés rencontrées	8
3.4	Séquençage temporel	9
3.4.1	Rappel de la Soutenance 1	9
3.4.2	Évolutions depuis la Soutenance 1	9
3.4.3	Difficultés rencontrées	9
3.5	Interface graphique	10
3.5.1	Rappel de la Soutenance 1	10
3.5.2	Évolutions depuis la Soutenance 1	10
3.5.3	Difficultés rencontrées	10
3.6	Gestion des fichiers et export audio	12
3.6.1	Rappel de la Soutenance 1	12
3.6.2	Évolutions depuis la Soutenance 1	12
3.6.3	Ce qui n’a pas pu être fait	12
3.6.4	Difficultés rencontrées	12
4	Bibliographie	14

1 Rôles et missions des membres

1.1 Alexandre – Moteur DSP & synthèse sonore

Alexandre se concentre sur le moteur DSP, domaine exigeant une maîtrise approfondie du traitement du signal et des mathématiques associées. Son travail englobe l'implémentation des oscillateurs, des enveloppes ADSR, des filtres numériques, la gestion de la polyphonie ainsi que les effets audio. Il est également responsable de l'architecture du thread audio temps réel, de la gestion des buffers circulaires, du passage en stéréo de toute la chaîne de signal, du modèle physique Karplus-Strong pour le piano et des trois effets audio : delay, réverbération de Schroeder et chorus.

1.2 Quentin – Séquençage & structures de données temporelles

Quentin prend en charge la logique du séquenceur et les structures de données temporelles. Cette responsabilité inclut la représentation interne des événements musicaux, les algorithmes de planification des steps, la synchronisation avec le moteur audio ainsi que la gestion des notes de type MIDI. La coordination entre le séquenceur et le moteur audio constitue un point critique nécessitant une collaboration étroite avec Alexandre.

1.3 Léa – Interface graphique

La développe l'interface graphique, tâche complexe impliquant la création d'éditeurs visuels pour la timeline, les paramètres d'instruments et les contrôles de transport. Son travail repose sur la bibliothèque `egui` (Immediate Mode GUI). L'ergonomie et la réactivité de l'interface détermineront largement l'expérience utilisateur finale.

1.4 Hugo – Système de fichiers & export audio

Hugo assure la gestion du système de fichiers, l'import et l'export au format JSON pour la persistance des projets, ainsi que l'exportation audio au format WAV via la bibliothèque `hound`. Cette couche inclut également la gestion des erreurs d'entrée/sortie et la validation des formats de fichiers pour garantir la robustesse de l'application.

2 Objectifs, planification et état d'avancement

2.1 Objectifs

La première soutenance visait à démontrer la viabilité technique du projet avec un prototype fonctionnel capable de générer et de lire du son. Les objectifs fixés dans le cahier des charges étaient les suivants :

- Moteur audio opérationnel (thread dédié, communication avec `cpal`, buffers circulaires stables) ;
- Synthèse de base fonctionnelle (oscillateurs simples, enveloppes ADSR) ;
- Séquenceur permettant le déclenchement programmatique de notes ;
- Interface graphique minimale (play, visualisation du piano) ;
- Export WAV fonctionnel via `hound`.

Pour la soutenance finale, les objectifs complémentaires étaient :

- Amélioration du rendu sonore des instruments (modèle physique Karplus-Strong) ;
- Passage en stéréo de toute la chaîne de traitement ;
- Ajout de trois effets audio sélectionnables depuis l'interface ;
- Finalisation de l'interface graphique (alignement grille/piano, marqueurs) ;
- Amélioration de l'export WAV (polyphonie, fidélité du rendu).

2.2 Planning

Tâche	Début prévu	Fin prévue	Responsable
Moteur audio	17/01/2026	07/03/2026	Alexandre
Oscillateurs & ADSR	17/01/2026	14/03/2026	Alexandre
Instruments & effets	01/04/2026	10/05/2026	Alexandre
Structures séquenceur	17/01/2026	11/03/2026	Quentin
Séquençage temporel final	01/04/2026	25/04/2026	Quentin
UI minimale (egui)	17/01/2026	13/03/2026	Léa
UI finale (egui + painter)	01/04/2026	10/05/2026	Léa
Export WAV	17/01/2026	14/03/2026	Hugo
Export WAV polyphonique	01/04/2026	10/05/2026	Hugo
Site web	17/01/2026	10/05/2026	Quentin

TABLE 1 – Planning global du projet

2.3 État d'avancement réel

Tâche	Objectif S1	Réel S1	Réel Final
Moteur Audio (DSP)	100%	100%	100%
Synthèse d'instruments	20%	30%	100%
Effets audio	0%	0%	100%
Séquençage temporel	50%	75%	100%
Interface graphique	40%	60%	100%
Système de fichiers	10%	20%	100%
Site Web	40%	70%	100%

TABLE 2 – Avancement par composant

3 Difficultés, choix technologiques et algorithmiques

3.1 Moteur audio temps réel

Le moteur audio constitue le noyau du système. Il s’articule autour de trois couches distinctes : la couche matérielle gérée par `cpal`, le séquenceur (`StepSequencer`) qui orchestre les pistes, et les moteurs d’instruments (`InstrumentEngine`) qui produisent les échantillons.

3.1.1 Choix technologiques

Le point d’entrée public est `play.rs`, qui expose des fonctions simples (`play`, `play_chord`, `play_melody`) au reste de l’application. Toute la configuration de `cpal`, la gestion des threads et le câblage du séquenceur sont encapsulés dans ce fichier.

La bibliothèque `cpal` est configurée avec une fréquence d’échantillonnage de 44 100 Hz, un buffer de 1 024 échantillons et une sortie stéréo deux canaux. Une closure appelée 44 100 fois par seconde depuis un thread système dédié appelle `StepSequencer::next_sample()` et copie le résultat sur les deux canaux.

3.1.2 Choix algorithmiques

La callback ne pouvant ni bloquer ni allouer de mémoire dynamique, toutes les structures sont pré-allouées à l’initialisation. À chaque appel de `next_sample()`, le séquenceur avance d’un échantillon, déclenche les `note_on` et `note_off` aux bons instants, puis délègue la génération du son à chaque `InstrumentEngine`. Chaque piste possède son propre engine, qui agrège un `VoicePool` de 32 voix et une `EffectsChain`. Les échantillons de toutes les pistes sont ensuite sommés.

La communication entre le thread audio et le thread principal repose sur un `Arc<Mutex<StepSequencer>`. Bien qu’un vrai DAW utiliserait des ring buffers SPSC lock-free, cette approche est fonctionnelle dans le cadre du projet.

3.1.3 Évolution : passage en stéréo

La principale évolution architecturale est le passage en stéréo de toute la chaîne de signal. La fonction `next_sample()` retourne désormais un tuple `(f32, f32)` à chaque niveau : oscillateur, voix, pool, effects chain et séquenceur. Le callback `cpal` écrit explicitement `frame[0] = l` et `frame[1] = r`.

3.1.4 Difficultés rencontrées

La principale difficulté concerne le paradigme callback-based imposé par `cpal`, qui contraint fortement la structure du code. Des problèmes de synchronisation entre le thread audio et le thread principal ont également été rencontrés. La propagation du type `(f32, f32)` à travers toutes les couches a nécessité de mettre à jour `build_fx()` dans `instruments.rs`. La race condition lors des modifications de l’interface (`import`, `clear track`) a été résolue en appelant `stop_playback()` avant toute modification des données partagées.

3.2 Synthèse sonore

3.2.1 Chaîne de signal

Chaque voix active suit une chaîne de traitement fixe :

Oscillateur → Filtre biquad (L/R) → Enveloppe ADSR → sortie (L, R)

La sortie de chaque voix est limitée par un soft clipping cubique ($y = x - x^3/3$) qui remplace le `tanh()` de la Soutenance 1, plus transparent et moins colorant. Toutes les voix actives sont sommées et multipliées par `MASTER_GAIN = 0.25` pour éviter les débordements de dynamique.

3.2.2 Génération de formes d’ondes : PolyBLEP

La génération naïve de formes d’ondes non sinusoïdales introduit de l’aliasing : les discontinuités abruptes produisent des composantes au-delà de la fréquence de Nyquist ($f_s/2$), qui se replient dans le spectre audible. La technique **PolyBLEP** corrige ce problème en ajoutant un polynôme local autour de chaque discontinuité. Le calcul est minimal (une à deux vérifications de phase par échantillon), adapté au traitement temps réel.

L’oscillateur repose sur une architecture **triple unisson** : trois oscillateurs légèrement désaccordés ($\pm 0.2\%$) sont répartis dans l’espace stéréo. L’oscillateur central est partagé à 0.5 sur les deux canaux, les oscillateurs 2 et 3 alimentent respectivement gauche et droit à 0.7. Quatre formes d’ondes sont disponibles : sinusoïdale, dents de scie, carrée et triangulaire (dérivée par intégration d’une onde carrée).

3.2.3 Karplus-Strong – modèle physique de corde

En Soutenance 1, le piano utilisait une onde triangulaire passée dans un filtre passe-bas à 4 kHz, sonnant comme un synthétiseur. Le remplacement par le modèle de Karplus-Strong constitue la principale amélioration sonore depuis la Soutenance 1.

Un buffer circulaire de longueur $N = \lfloor f_s/f_0 \rfloor$ représente un cycle de vibration de la corde. À chaque échantillon, on filtre par un filtre passe-bas du premier ordre, puis on réécrit la valeur filtrée :

$$y[n] = \frac{y_{\text{buf}}[n] + y_{\text{prev}}}{2} \times d, \quad d = \left(1 - \frac{f_0}{80\,000}\right)_{[0.990, 0.9995]}$$

Les hautes fréquences s’amortissent plus vite que les basses, reproduisant le comportement d’un piano acoustique : A0 (27 Hz) sonne environ 8 secondes, C8 (4186 Hz) s’éteint en moins d’une seconde. Le buffer est initialisé à chaque `note_on` avec du bruit blanc filtré passe-haut simulant le marteau.

3.2.4 Enveloppes ADSR

L’enveloppe ADSR modélise l’évolution temporelle de l’amplitude selon quatre phases. Plutôt que des rampes linéaires, nous utilisons des courbes exponentielles via un filtre one-pole :

$$\text{level}[n] = \text{target} + (\text{level}[n-1] - \text{target}) \times e^{-1/(t_s \cdot f_s)}$$

Ce comportement correspond à la perception humaine de l’intensité sonore. Une machine à états explicite (`Idle`, `Attack`, `Decay`, `Sustain`, `Release`) gère les transitions et signale quand la voix peut être libérée. Pour le piano, `sustain_level = 0.0` : le son décroît toujours vers le silence, reproduisant le comportement acoustique réel.

3.2.5 Filtrage numérique : filtre biquad

Le filtre biquad IIR du second ordre implémente l'équation aux différences :

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

Les cinq coefficients sont calculés selon les formules de l'*Audio EQ Cookbook* de Bristow-Johnson via la transformation bilinéaire : 5 multiplications et 4 additions par échantillon. Lors d'un changement de paramètres, les coefficients sont interpolés linéairement sur 64 échantillons pour éviter les clics. Chaque voix possède deux filtres indépendants (`filter_l` / `filter_r`).

3.2.6 Polyphonie et voice stealing

La polyphonie repose sur un pool de 32 voix pré-allouées. Si toutes sont actives, la stratégie de voice stealing sélectionne en priorité une voix en phase Release (son extinction est peu perceptible), puis en dernier recours la voix la plus ancienne (champ `age` incrémenté à chaque échantillon).

3.2.7 Difficultés rencontrées

La principale difficulté a été l'identification de la cause des sons incohérents observés en Soutenance 1. Le `tanh()` introduisait une distorsion harmonique sur tous les instruments. La mise en place de Karplus-Strong a nécessité une gestion fine du buffer circulaire, notamment le calcul correct de la période en samples et l'initialisation propre à chaque `note_on`. Un bruit déterministe (hash de position XOR) a été préféré au bruit aléatoire pour garantir la reproductibilité.

3.3 Effets audio

3.3.1 Architecture

Les effets sont organisés autour d'un enum `EffectMode` (`None` / `Reverb` / `Delay` / `Chorus`) et d'un champ `wet_mix` global. La fonction `process()` de `EffectsChain` dispatche vers l'effet actif uniquement, évitant l'application systématique de toute la chaîne. Les trois effets sont sélectionnables depuis l'interface via un `ComboBox`, avec un slider de dosage.

3.3.2 Delay

Le delay utilise un buffer circulaire de deux secondes maximum. Le signal retardé est réinjecté en entrée avec un coefficient de feedback, créant des échos répétés qui décroissent naturellement. Le mix sec/mouillé est contrôlable indépendamment.

3.3.3 Réverbération de Schroeder

La réverbération implémente le réseau de Schroeder (1962) : quatre filtres en peigne parallèles (délais 1601, 1861, 2351, 2503 samples) simulent les premières réflexions, puis deux filtres passe-tout en série diffusent le signal. Les délais ont été mis à jour depuis la Soutenance 1 (anciens : 1557, 1617, 1491, 1422) pour réduire les résonnances périodiques. Bien que plus simple que des algorithmes modernes comme `Freeverb`, cette approche produit une réverbération musicalement utilisable avec un coût computationnel très faible.

3.3.4 Chorus

Le chorus est l'effet ajouté depuis la Soutenance 1. Il duplique le signal dans un buffer circulaire dont la position de lecture est modulée par un LFO. Deux LFO déphasés de 90° alimentent les canaux gauche et droit, produisant un élargissement stéréo naturel :

$$\text{delay}_L = d_{\text{base}} + A \cdot \sin(2\pi f_{\text{LFO}} \cdot t), \quad \text{delay}_R = d_{\text{base}} + A \cdot \sin\left(2\pi f_{\text{LFO}} \cdot t + \frac{\pi}{2}\right)$$

3.3.5 Difficultés rencontrées

L'ajout du chorus a nécessité de mettre à jour `build_fx()` dans `instruments.rs` pour inclure les nouveaux champs `chorus`, `mode` et `wet_mix` dans `EffectsChain`, ce qui provoquait une erreur de compilation. L'intégration de l'enum `EffectMode` dans `play.rs` a également nécessité de propager le paramètre d'effet à travers toutes les fonctions publiques de l'API audio.

3.4 Séquençage temporel

3.4.1 Rappel de la Soutenance 1

Lors de la première soutenance, l’objectif principal était de développer un système de séquençage musical fonctionnant en temps réel. Le projet permettait de programmer des événements musicaux sous forme de grille, organisée en plusieurs pistes et plusieurs steps. Les fonctionnalités implémentées étaient la gestion de plusieurs pistes indépendantes, l’activation et la désactivation de steps, la gestion des notes MIDI, la synchronisation avec le BPM et les commandes de transport (play, stop).

L’architecture repose sur une séparation entre les données musicales (**Step** et **Track**), la logique temporelle (**StepSequencer**) et la génération audio (**InstrumentEngine**), avec un modèle sample-based garantissant la synchronisation précise avec le callback audio.

3.4.2 Évolutions depuis la Soutenance 1

Le step sequencer permet désormais l’ajout et la suppression dynamique de pistes, la modification du BPM pendant l’exécution, le redimensionnement du nombre de steps, la gestion du mute des pistes, la gestion des notes et vitesses par step, ainsi que la lecture en boucle configurable (`looping = false` pour la lecture unique).

La gestion temporelle a été finalisée : le calcul de la durée des steps est effectué automatiquement à partir du BPM, du nombre de steps et du sample rate. Une logique de gate gère automatiquement la durée des notes pour éviter les notes bloquées. Plusieurs tests unitaires ont été ajoutés pour vérifier le bon calcul de la temporalité, le fonctionnement des steps et le comportement du transport.

3.4.3 Difficultés rencontrées

La principale difficulté concerne la gestion de la temporalité en temps réel : convertir les informations musicales (BPM) en nombre d’échantillons pour déterminer précisément la durée de chaque step. Il fallait garantir que chaque `note_on` soit correctement associé à un `note_off` pour éviter les notes bloquées. La gestion des structures de données en Rust a demandé une attention particulière concernant les emprunts mutables et la prévention des accès hors limites.

Certaines limitations restent présentes : la durée des notes est fixe pour tous les steps, chaque piste reste monophonique côté séquenceur, et il n’existe pas encore de système de swing ou de micro-timing.

3.5 Interface graphique

3.5.1 Rappel de la Soutenance 1

Lors de la première soutenance, trois parties distinctes de l'interface étaient implémentées : en haut, la barre de menus pour les fichiers, la sélection des instruments et les commandes de transport ; à gauche, le piano permettant de tester les sons en cliquant sur les touches (réparti sur quatre pages) ; au centre, la grille de cases cochables représentant la partition.

Une difficulté importante avait été rencontrée lors de la création du piano : l'utilisation de boutons cliquables ne permettait pas le redimensionnement. La solution retenue a été d'utiliser des rectangles dessinés manuellement avec le `Painter` d'egui, cliquables via `allocate_rect`.

3.5.2 Évolutions depuis la Soutenance 1

Pour la barre du haut, un bouton de suppression de toutes les notes de la piste active (*Clear track*) a été ajouté, ainsi qu'un sélecteur d'effet fonctionnel (ComboBox `None` / `Reverb` / `Delay` / `Chorus`) accompagné d'un slider de dosage wet. Deux boutons de défilement de partition ont également été intégrés.

Pour la grille centrale, les cases n'étaient pas alignées avec le piano et il n'existait pas de marqueur de temps. Ces points ont été corrigés : des barres ont été ajoutées au-dessus de la partition (une barre = une seconde), les cases sont désormais alignées avec le piano, et le nom de la note apparaît dans les cases sélectionnées.

3.5.3 Difficultés rencontrées

Le borrow checker d'egui impose une contrainte importante : `ui.painter()` (borrow immutable) et `ui.allocate_rect()` (borrow mutable) ne peuvent pas coexister sur le même `ui`. La solution adoptée est une architecture en deux passes : toutes les interactions (`allocate_rect`) sont traitées en premier, puis le dessin (`painter`) est effectué en second.

Pour le défilement de la partition, l'approche initiale envisageait une barre mobile horizontale. Cette solution s'est avérée incompatible avec les rectangles fixes utilisés pour les cases. Des boutons de navigation ont donc été adoptés.

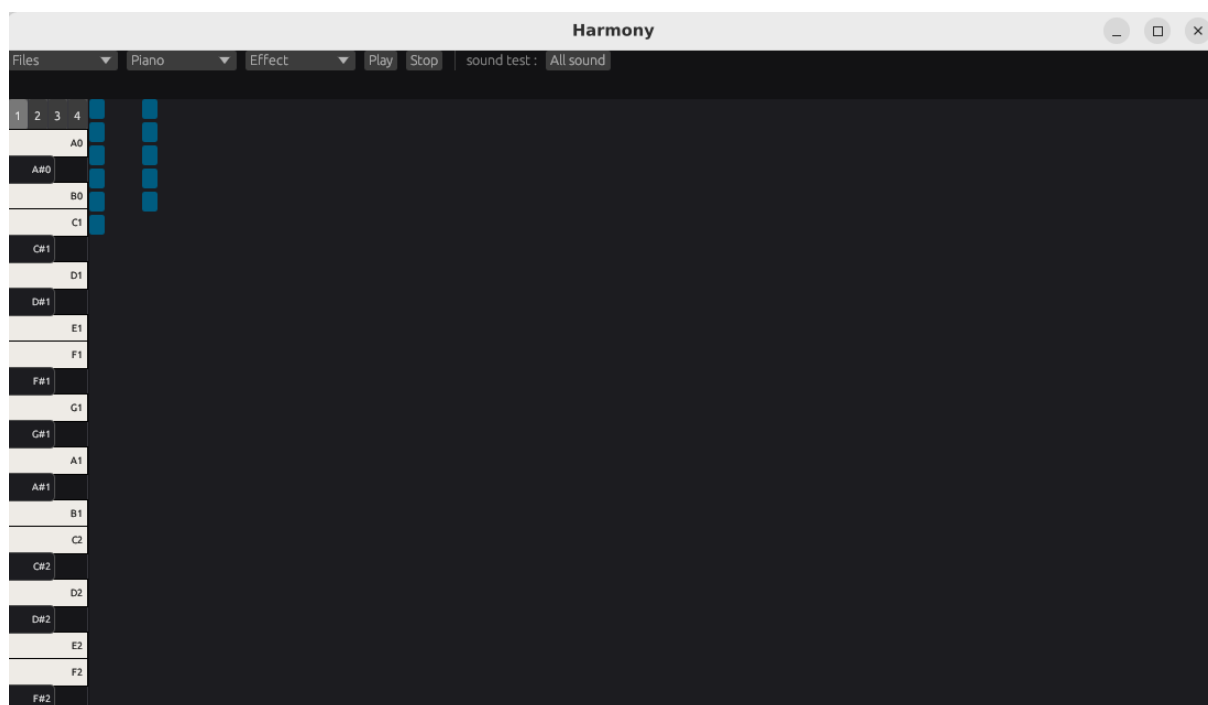


FIGURE 1 – Ancienne Interface

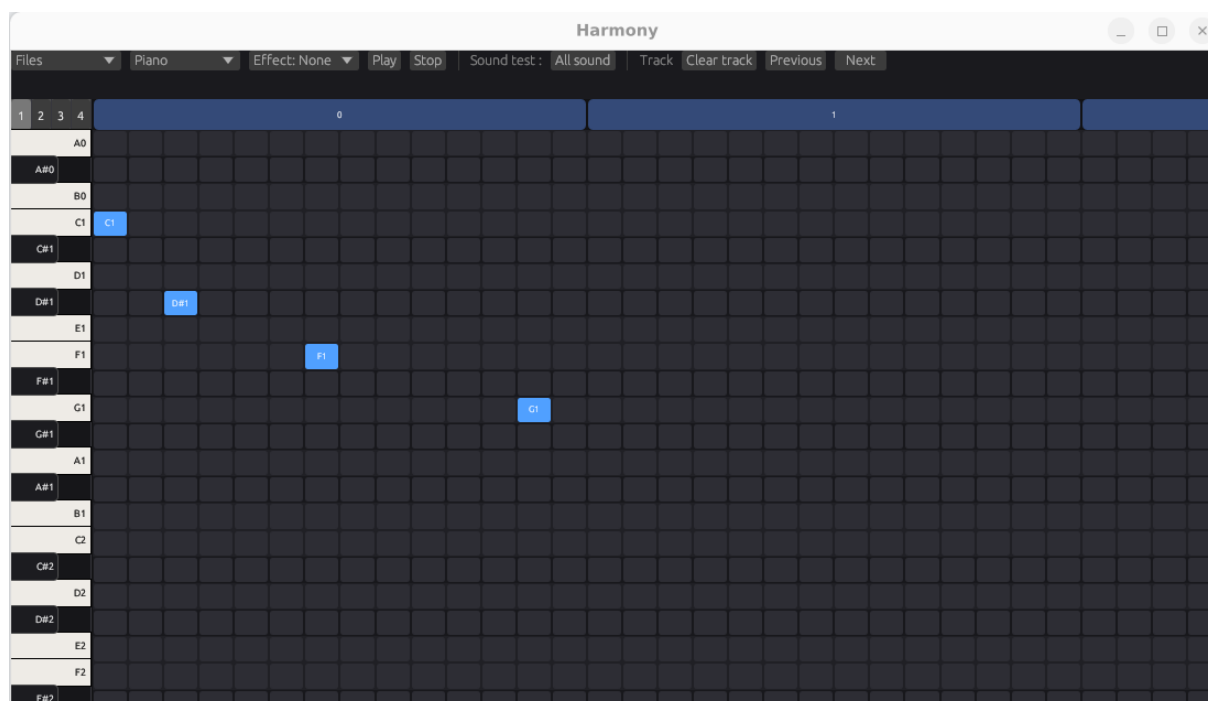


FIGURE 2 – Nouvelle Interface

3.6 Gestion des fichiers et export audio

3.6.1 Rappel de la Soutenance 1

Lors de la première soutenance, la sauvegarde au format JSON et l’export WAV étaient fonctionnels. Ces fonctionnalités permettaient de conserver les compositions et de générer un fichier audio exploitable. Cependant, l’audio exporté ne correspondait pas exactement au son entendu dans l’application : le pipeline d’export reposait sur une synthèse simple qui ne reproduisait pas entièrement le comportement du moteur audio. Le mixage entre plusieurs pistes et la synchronisation des événements restaient peu optimisés.

3.6.2 Évolutions depuis la Soutenance 1

Pour cette soutenance finale, le travail a été divisé en deux parties : l’export individuel de chaque piste et la gestion de la polyphonie.

L’implémentation de l’export a été entièrement revue. Au lieu de se baser sur les données JSON pour effectuer une nouvelle synthèse, le moteur audio utilisé pour la lecture en temps réel est désormais réutilisé directement, le flux étant redirigé vers un fichier plutôt que vers la sortie audio. Cette méthode assure une cohérence entre le rendu entendu dans l’application et le fichier exporté.

Pour la polyphonie, les différents flux audio sont combinés lors de l’export, produisant un signal final unique. Une attention particulière a été portée à la synchronisation des événements et à la gestion des amplitudes pour éviter les saturations.

3.6.3 Ce qui n’a pas pu être fait

L’export au format MP3, envisagé après la première soutenance, a été abandonné. Après plusieurs essais, le résultat obtenu n’était pas suffisamment satisfaisant. L’export WAV étant pleinement fonctionnel et offrant une qualité non compressée, la valeur ajoutée du MP3 ne justifiait pas le surcoût de complexité.

3.6.4 Difficultés rencontrées

Le changement d’approche (de la synthèse JSON vers la réutilisation du moteur) a nécessité une restructuration importante du pipeline. La synchronisation entre les différentes pistes dans le cadre de la polyphonie a représenté une difficulté importante. Le mixage des pistes a demandé plusieurs ajustements pour éviter les problèmes de saturation et de déséquilibre de volume.

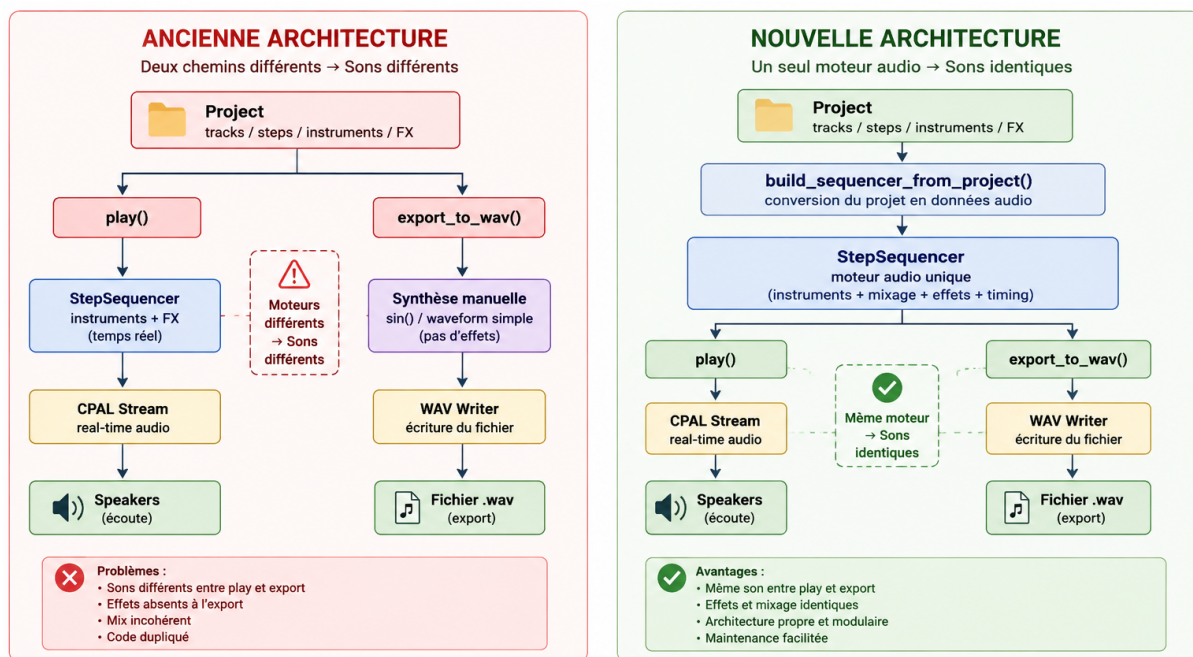


FIGURE 3 – Ancienne-Nouvelle Architecture

4 Bibliographie

1. cpal – Cross-Platform Audio Library. <https://github.com/RustAudio/cpal>
2. Julius O. Smith III, *Introduction to Digital Filters with Audio Applications*, CCRMA, Stanford University, 2007. <https://ccrma.stanford.edu/~jos/filters/>
3. Nils Jonas Norberg, *PolyBLEP Oscillator*, 2010.
4. Kevin Karplus & Alex Strong, *Digital Synthesis of Plucked-String and Drum Timbres*, Computer Music Journal, 1983.
5. M. R. Schroeder, *Natural Sounding Artificial Reverberation*, Journal of the Audio Engineering Society, 1962.
6. Robert Bristow-Johnson, *Audio EQ Cookbook*. <https://www.w3.org/TR/audio-eq-cookbook/>
7. egui – Immediate Mode GUI in Rust. <https://github.com/emilk/egui>
8. hound – WAV encoding/decoding in Rust. <https://crates.io/crates/hound>
9. Udo Zölzer (éd.), *Digital Audio Effects*, 2^e éd., Wiley, 2011.
10. ringbuf – Lock-free SPSC ring buffer. <https://crates.io/crates/ringbuf>